

RATE LIMITER SYSTEM DESIGN

- INTRODUCTION

- 1.Analogy
- 2.What is a Rate Limite
- 3.Why do we need a Rate Limite

- DECIDING REQUIREMENTS

- 4.Functional Requirements
- 5.Non Functional Requirements

- CAPACITY ESTIMATION

- 6.DAU-MAU
- 7.Throughput
- 8.Storage and Memory
- 9.Network and Bandwidth Estimation

- HIGH LEVEL DESIGN

- 10.HIGH LEVEL DESIGN:Placemet of a Ratelimiter
- 11.HIGH LEVEL DESIGN:Basic High Level Flow
- 12.HIGH LEVEL DESIGN:Storing Request Counters
- 13.HIGH LEVEL DESIGN:Storing Ratelimiter Rules

- DEEP DIVE INSIGHTS

- 14.Deep Dive Insights:Tocken Bucket Algorithm
- 15.Deep Dive Insights:Leaky Bucket Algorithm
- 16.Deep Dive Insights:Fixed Window Counter Algorithm

- 17.Deep Dive Insights:Sliding Window Log Algorithm
 - 18.Deep Dive Insights:Sliding Window Counter Algorithm
 - 19.Deep Dive Insights:Handling Race Condition
-

INTRODUCTION

Analogy

Imagine you're at an amusement park with a special roller coaster that everyone wants to ride.

To be fair and give everyone a chance, the park has a rule: **each person can ride the roller coaster only twice a day.**

To enforce this rule, a security guard is stationed at the entrance to make sure no one rides more than twice.

This way, no one can ride it too many times, and more people can enjoy it.

This rule, enforced by the security guard, is like a real-life example of a **rate limiter**.



What is a Rate Limiter

In system design, a rate limiter works like that security guard.

Just how a security guard ensures that no one rides the roller coaster more than twice a day, a rate limiter ensures no user can make more requests than allowed in a specific time period.

Example

Let's say on Instagram, no user can post more than **100 times a day**.



Rate Limiter

This prevents any one user from overwhelming the system and helps keep everything running smoothly.

Why do we need a Rate Limiter

Rate-Limiter plays a critical role because of the following reasons:

1. Prevent Overloading

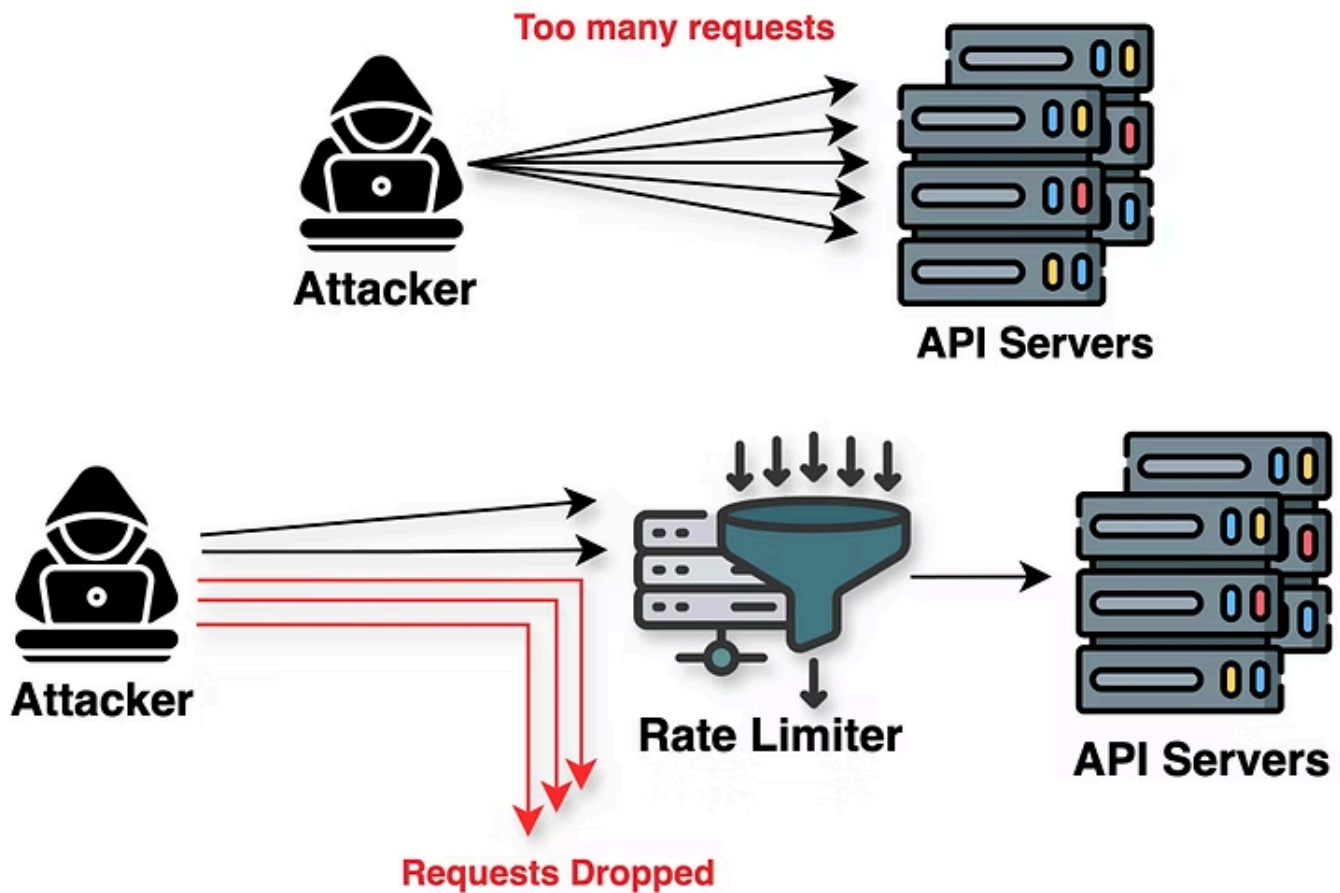
To stop the system from being overwhelmed by too many requests at once.

2. Ensure Fair Usage

One user sending too many requests can block other users from using the service. Hence, Rate-Limiter is essential to make sure all users get a fair share of system resources.

3. Control Costs

An overwhelming amount of requests also means an overwhelming amount of costs to process them. Therefore, Rate-Limiter is essential to prevent high costs.



DECIDING REQUIREMENTS

Functional Requirements

Requirement	Description
Rate Limiting	Our system should be able to rate limit the requests. The rate limiting could be based on either property - IP address, user ID, device ID, or any other properties.
User Notification	Our system should inform users when they are throttled for exceeding the limit.

Non Functional Requirements

Requirement	Description
Availability	The system should be highly available - 99.99999% uptime
Low Latency	Rate Limiting System is implemented as a part (sub-system) of a big system. Therefore, it needs to have very low latency; otherwise, it will slow down the overall system.
Cost Effectiveness	The Rate Limiting System should be cost-effective; otherwise, it will increase the overall cost of the big system.

CAPACITY ESTIMATION

DAU MAU

How many users will be using your software?

Metric	Value
Daily Active Users (DAU)	500 million
Monthly Active Users (MAU)	2 billion

Throughput

Let's calculate how many rate-limited requests we need to handle.

Assume each user makes an average of **100 API requests per day**.

Calculation

Users (DAU)	Requests per User per Day	Total Requests per Day
500 million	100	50 billion

Storage and Memory

For Rate-Limiting, we mainly store:

- **Counter Data** – The count of each type of request.

Counter Data Table

User	Request Type	Current Count
User1	No. of Posts/Day	4
User1	No. of Friend Requests/Hour	3
User2	No. of Posts/Day	3

Storage Calculation

- Let's assume, on average, one entry in the **Counter Table** = **100 bytes**.
- Our system supports **50 different APIs** (Request Types).
- For every user, we store the current count for each type of request.

Total Storage Calculation:

Total storage = Number of Entries × Size of Entry

= (500 million users × 50 Request Types) × 100 bytes

= 2.5 TB

Since the latency requirement is low and the storage size is **2.5 TB** (manageable for a caching solution), we can use **cache** to store the entire counter data.

Network and Bandwidth

Network / Bandwidth estimation tells us how much data flows in and out of our system per second.

To estimate bandwidth, we need to consider the data transfer for requests and responses:

Parameter	Value
Average size of a request/response	1 KB (assuming a small payload for API calls)
Total daily data transfer	50 billion requests/day × 1 KB = 50 TB/day
Average bandwidth required	50 TB/day / 24 hours / 3600 seconds ≈ 579 GB/hour ≈ 161 MB/s

HIGH LEVEL DESIGN

HIGH LEVEL DESIGN :Placement of a RateLimiter

Now, before designing anything, the most important thing to decide is **where will our rate limiter system reside?**

Should it come **before the server** or **reside inside the server**?

Let's explore both scenarios:

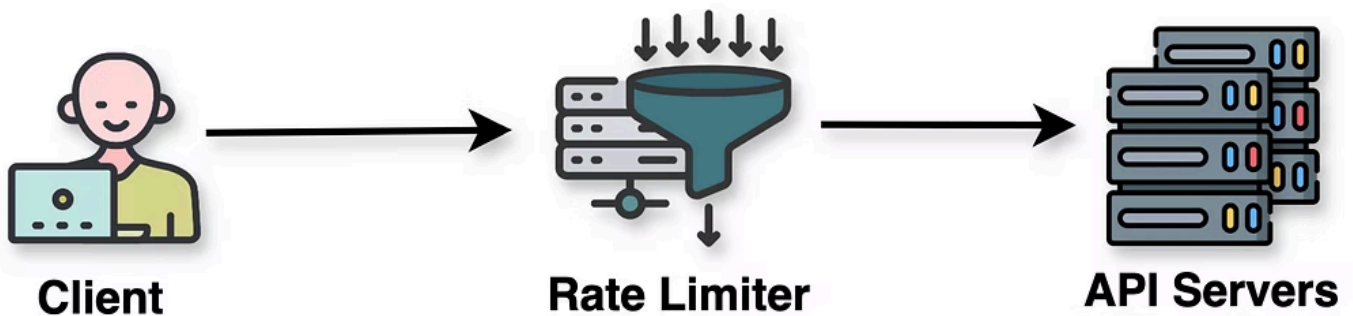
Scenario 1: Rate Limiter Comes Before the API Servers

Pros:

- **Enhanced Security:** Blocks malicious traffic early, improving security.
- **Reduced Load:** API servers handle less traffic as the rate limiter blocks many requests.

Cons:

- **Increased Latency:** Adds an extra processing step, which might slow things down.
- **Single Point of Failure (SPOF):** If the rate limiter fails, it could stop all traffic, potentially bringing down the whole system.



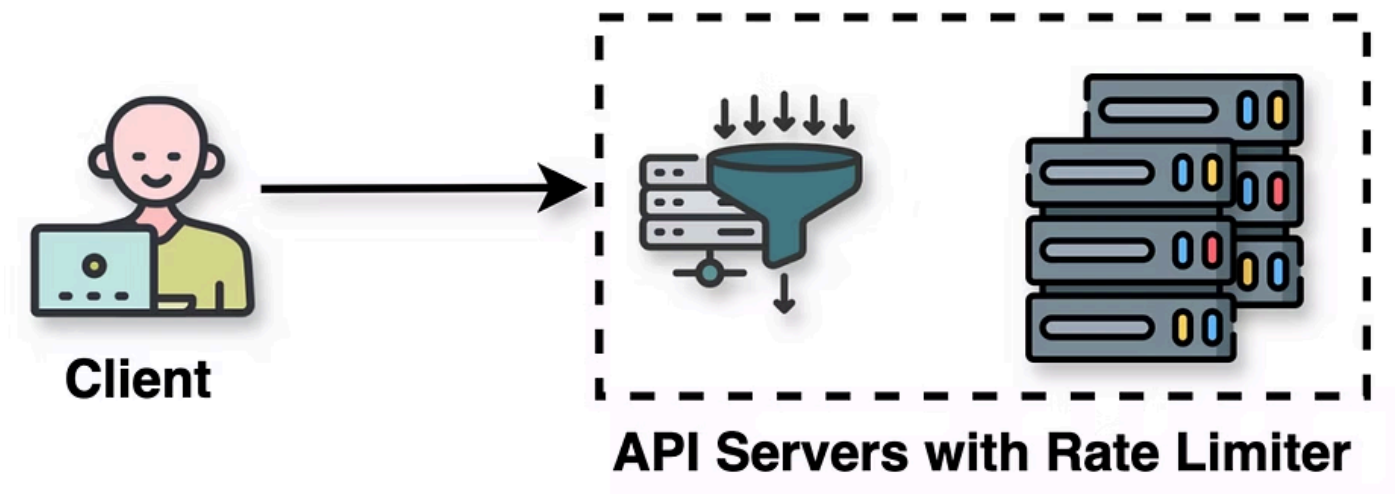
Scenario 2: Rate Limiter Resides at the API Servers

Pros:

- **Granular Control:** Allows custom rate limiting for each server or service.
- **No Single Point of Failure:** Improves fault tolerance as there is no central failure point.

Cons:

- **Increased Server Load:** Servers have to handle the rate limiting logic, adding extra work.



Recommendation

Choose based on your **tech stack and resources**:

- For **centralized control and security**, place the rate limiter **before the API servers**.
- For **granular control and no central failure point**, use **server-side rate limiting**.

For our further discussion, we will be **placing our rate limiter before the API servers**.

HIGH LEVEL DESIGN :Basic High level flow

1. Client Request

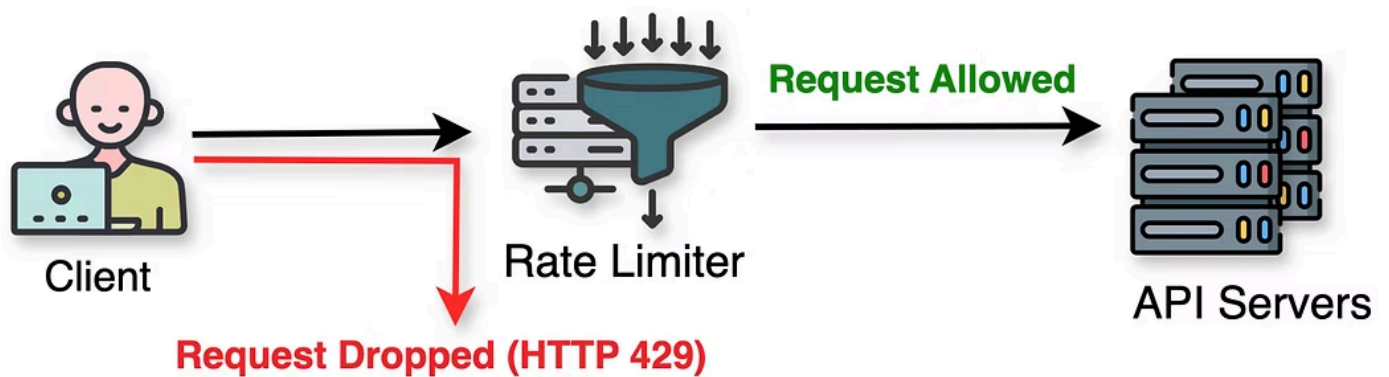
A client sends a request.

2. Rate Limiter Check

- When a request arrives, it first goes to the **Rate Limiter**.
- The Rate Limiter checks the request count:
 - If the count is too high, the request is **dropped**.
 - If the count is within limits, the request is **forwarded** to the API server.

3. API Server

- If the request is allowed, it reaches the **API Server** for processing.



◆ Fun Fact

When a request exceeds the rate limit, the **Rate Limiter** sends an **HTTP 429 (Too Many Requests)** response back to the client.

This status code indicates that the user has sent too many requests in a given amount of time and should **try again later**.

HIGH LEVEL DESIGN :Storing Request Counters

How Does the Rate Limiter Work?

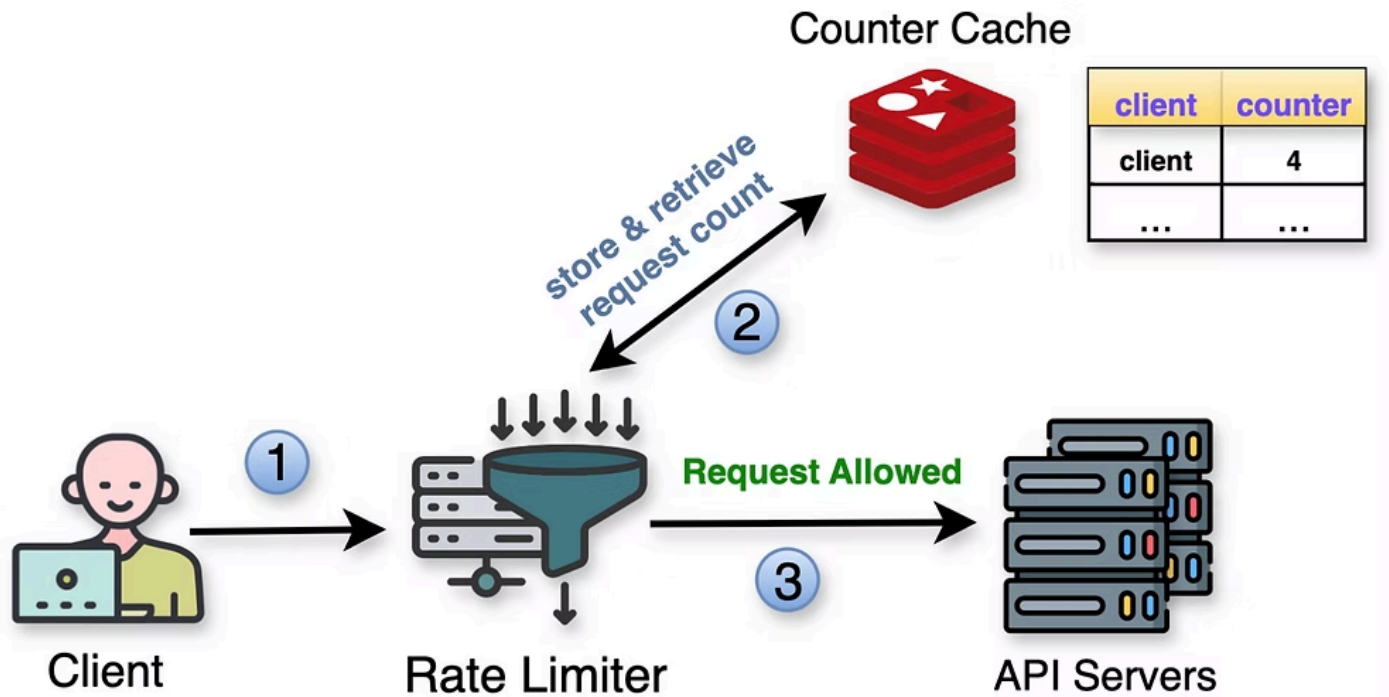
When a client sends a request, the request first reaches the rate limiter. The rate limiter determines whether to allow or drop the request by storing the request count.

- **Allowed Request:** If the count is within the limit, the request is forwarded to the API servers.
- **Dropped Request:** If the count exceeds the limit, the request is rejected.

Where Do We Store the Count?

Storing this count efficiently is crucial. Here's why:

- **Database Storage:** Not ideal because databases are too slow for this use case.
- **Cache Storage (Redis):** A much better option because:
 - It is significantly faster.
 - The data being stored (request count) is small and lightweight.



By using **Redis**, we ensure efficient and real-time request counting, enabling the rate limiter to perform effectively.

HIGH LEVEL DESIGN :Storing RateLimiter Rules

Let's say the client sends a request. The request reaches the rate limiter. Now, how does the rate limiter know whether to allow or drop the request?

By checking that the request count has not crossed the limit. We know that the counts are stored in the **counter cache**, but where is the limit stored? The limit is stored in the **Rules DB**. The **Rules Service** updates these rules in the **Rules Cache** periodically for faster access.

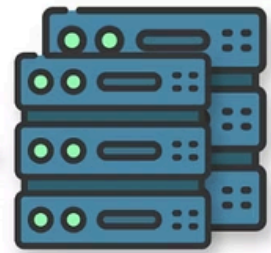
Rules Service updates
cache periodically

Rules Cache



2

Cache Update



Rules Service

Fetches Rate
Limiting Rules



1

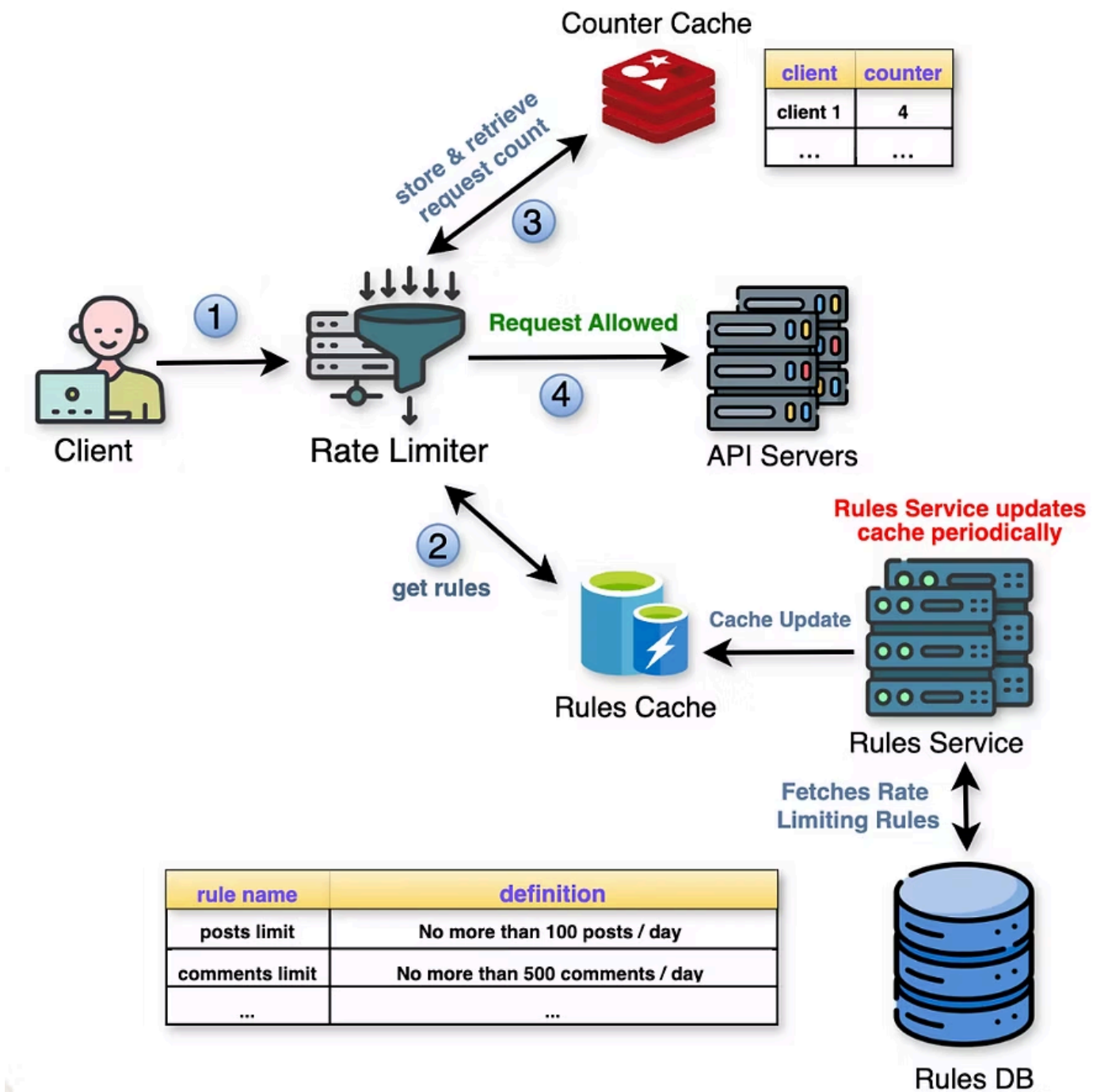


Rules DB

rule name	definition
posts limit	No more than 100 posts / day
comments limit	No more than 500 comments / day
...	...

Overall Flow:

1. The request is sent to the rate limiter.
2. The **Rate Limiter** gets the rate-limiting rules from the **Rules Cache**.
3. The **Rate Limiter** gets the request count from the **Counter Cache** (Redis).
4. **If the request count is within the allowed limit**, the request is forwarded to the API servers.
5. **If the request count exceeds the limit**, the request is denied.



DEEP DIVE INSIGHTS

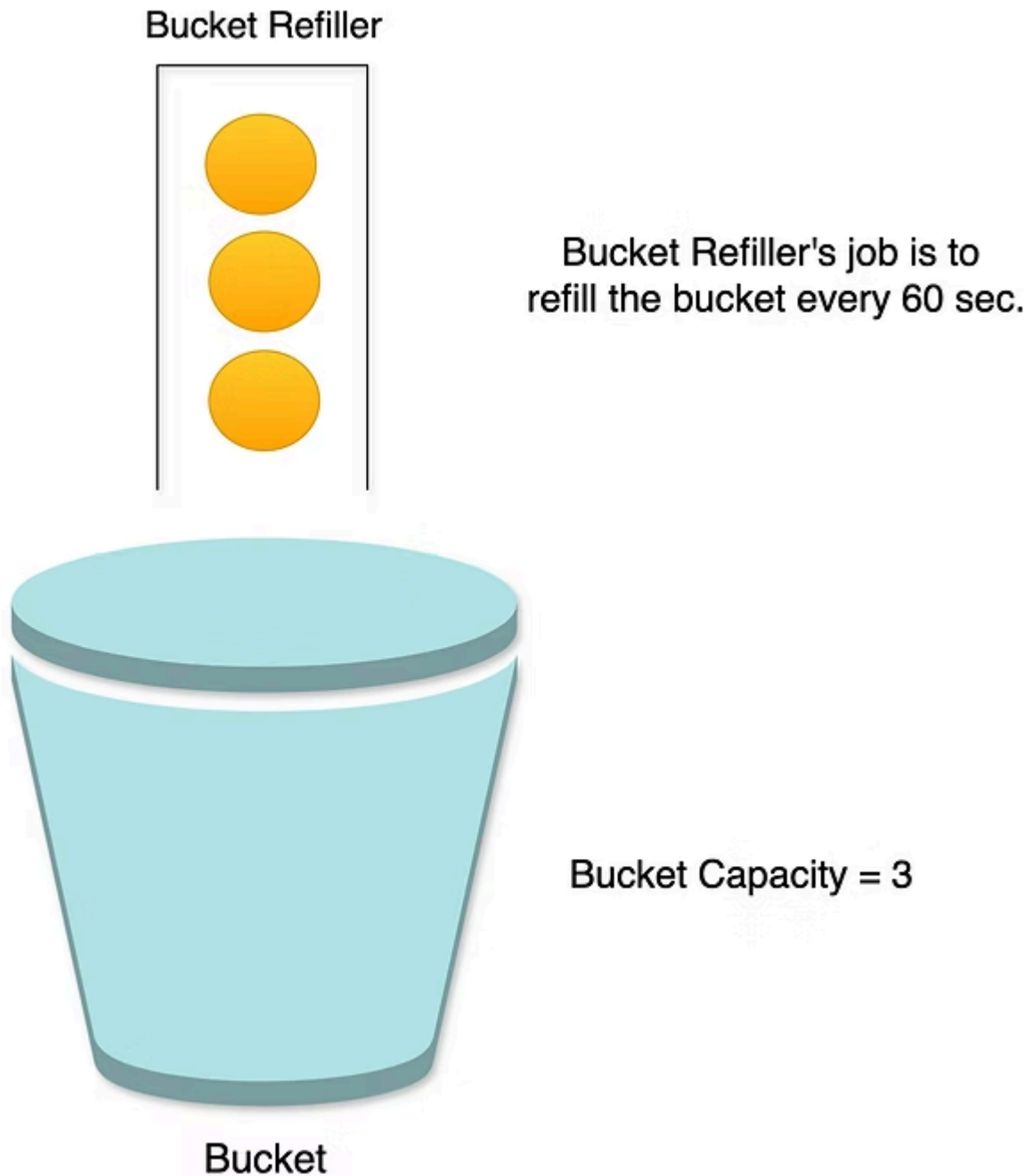
DEEP DIVE INSIGHTS :Token Bucket Algorithm

The Token Bucket Algorithm is very simple to understand and also effective. It is used by popular companies such as Amazon while implementing their rate-limiting systems.

Overview

There are two main components of the algorithm:

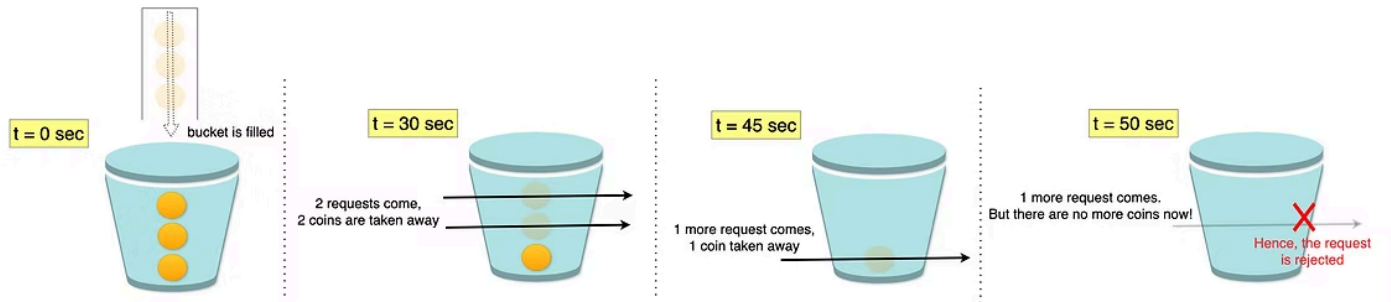
- **Bucket:** A bucket is a container that can store coins.
 - The maximum number of coins that can be stored in the bucket = **Bucket Capacity**
- **Bucket Refiller:** The refiller replenishes the bucket at a particular rate.
 - Example: Every **60 seconds**



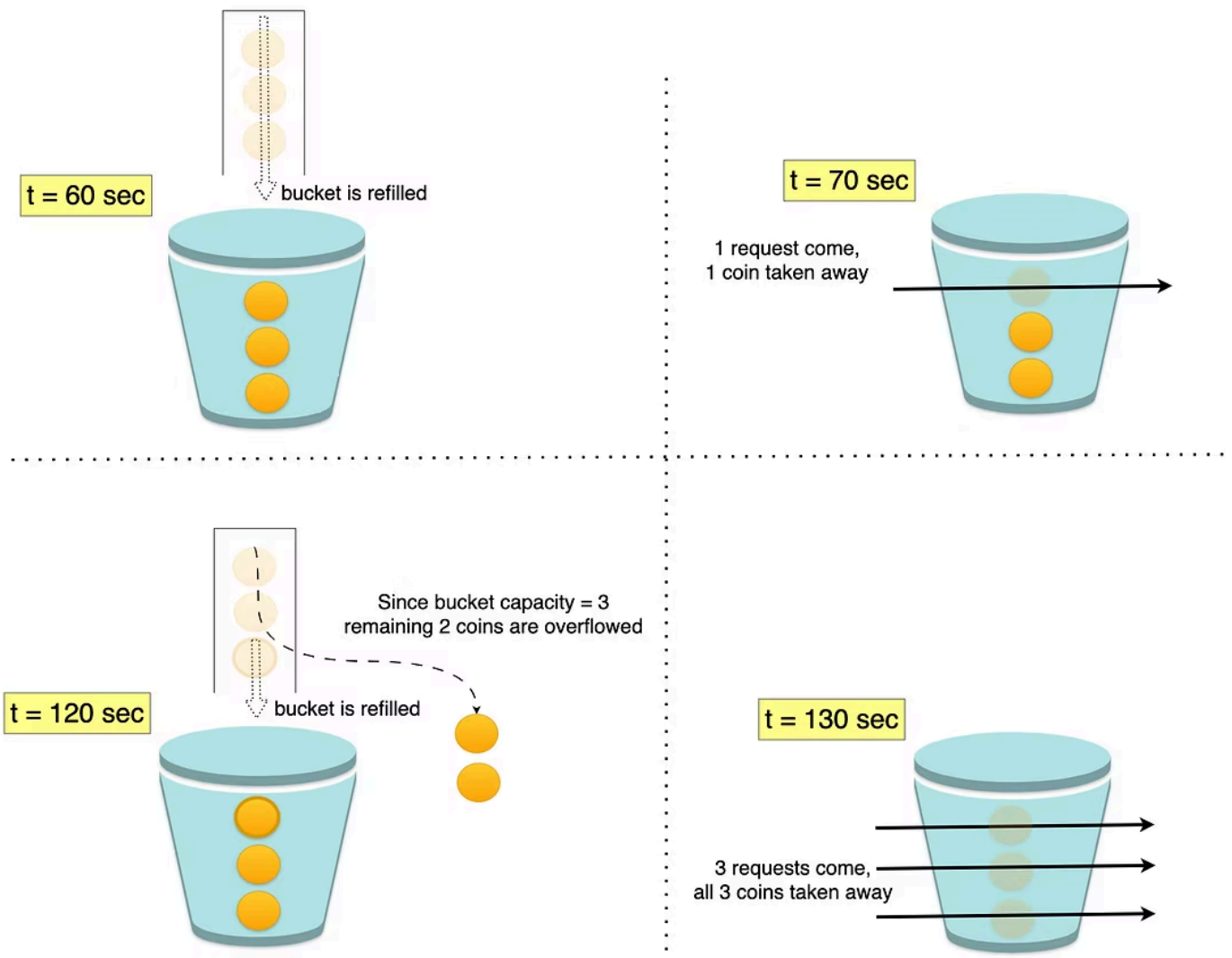
Working Mechanism

The following diagram explains how the algorithm works. Follow the diagram chronologically (**from $t = 0 \rightarrow 130$**) to get a complete understanding.

- A request needs to **consume 1 coin** to pass through.
- The **max number of coins** a bucket can hold is **3**.
- When the bucket gets empty, **no more requests** are able to pass through (**$t = 50$ sec**), until the bucket is refilled again.



- The bucket is refilled **every 60 sec** (**$t = 0$ sec, $t = 60$ sec, $t = 120$ sec**).
- Any **surplus is overflowed** (**$t = 120$ sec**).



Pros and Cons

Pros	Cons
Simple & intuitive to understand & implement.	Challenging to choose the right parameters (bucket capacity and refill rate).
Memory efficient. Only two parameters - bucket capacity and refill rate.	
Allows traffic burst or 'sudden' traffic. Example: At $t = 130$ seconds, 3 requests arrive, and all 3 coins are used up, allowing the requests to go through.	
This means the system can handle a sudden increase in traffic if there are enough coins available.	

DEEP DIVE INSIGHTS :Leaky Bucket Algorithm

The most intuitive way to understand the Leaky Bucket Algorithm is to visualize a leaky bucket.



Analogies:

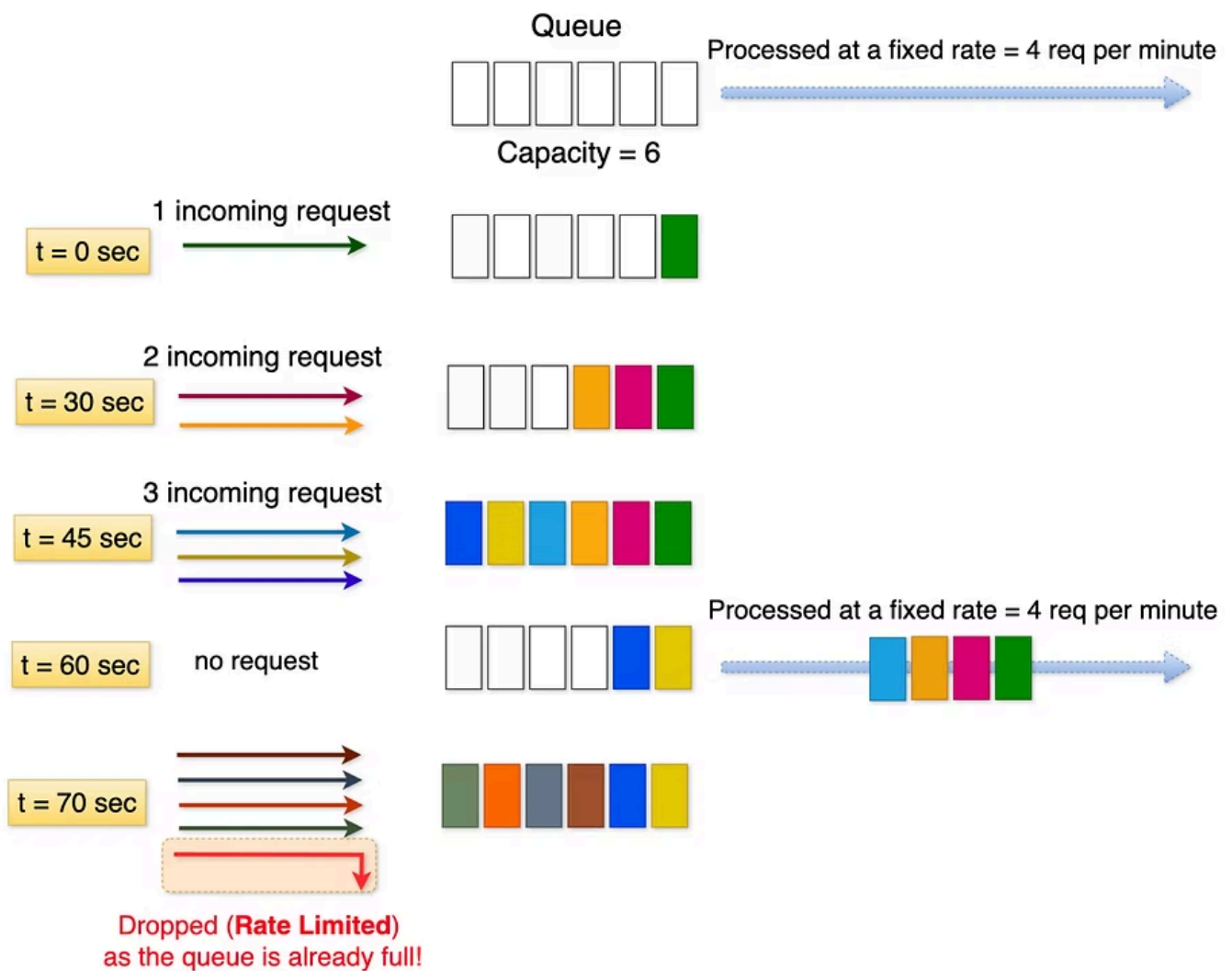
- **Incoming water (tap)** represents all the incoming requests to our system.
- **Outgoing water (via hole)** represents all the requests that are processed.
- When the tap water flows in too quickly, the bucket overflows.
 - The overflowing water represents all the requests that are dropped and rate-limited.
- **Bucket** represents a processing queue.



Understanding the Algorithm:

Now, refer to the following diagram to understand the algorithm better:

- The queue can hold up to **6 requests** at a time.
- When a request arrives:
 - It is added to the queue **if there is space**.
 - Otherwise, it is **dropped** (e.g., at **t = 70 sec**).
- All queued requests are **processed in order** (not rate-limited).
 - **Processing rate = 4 requests per 60 sec.**



DEEP DIVE INSIGHTS :Fixed Window Counter Algorithm

The **Fixed Window Counter Algorithm** is very simple to understand.

How It Works

- Time is divided into fixed windows, such as **[0-60 seconds]**, **[60-120 seconds]**, and so on.
- A limit is set on how many requests can pass through in each window.
- If the number of requests exceeds the limit, the extra requests are **dropped (rate limited)** until the next window starts and the counter resets.



Example

In the diagram, the system allows **up to 5 requests per minute**.

At the start of each new minute, the counter **resets to 0**.

- If **≤ 5 requests** come in within a minute, they are processed normally.
- If **> 5 requests** arrive, the extra requests are **dropped** (unlike the Leaky Bucket Algorithm, where requests might be queued).

This ensures **efficient rate limiting** without carrying over excess traffic.

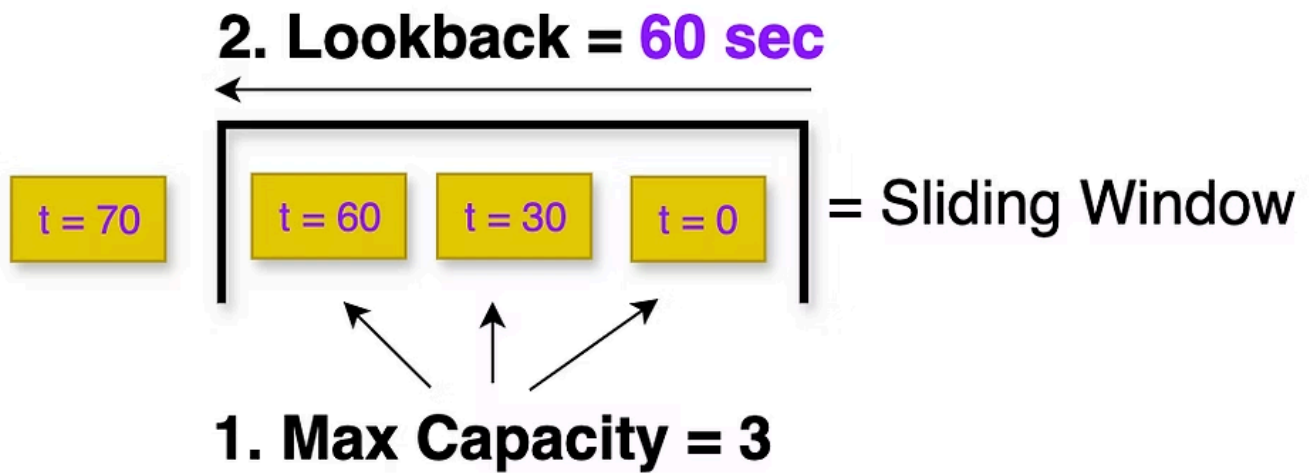
DEEP DIVE INSIGHTS :Sliding Window Log Algorithm

The **Sliding Window Log Algorithm** is an improvement over the **Fixed Window Counter Algorithm**. It keeps a sliding window instead of a fixed window, addressing the issue of sending too many requests at the edges of fixed windows.

Key Concepts

A window in this algorithm stores timestamps of the requests. There are two key components:

- **Sliding Window Capacity:** Maximum number of timestamps that the window can hold.
- **Sliding Window Lookback:** How many seconds to look back.



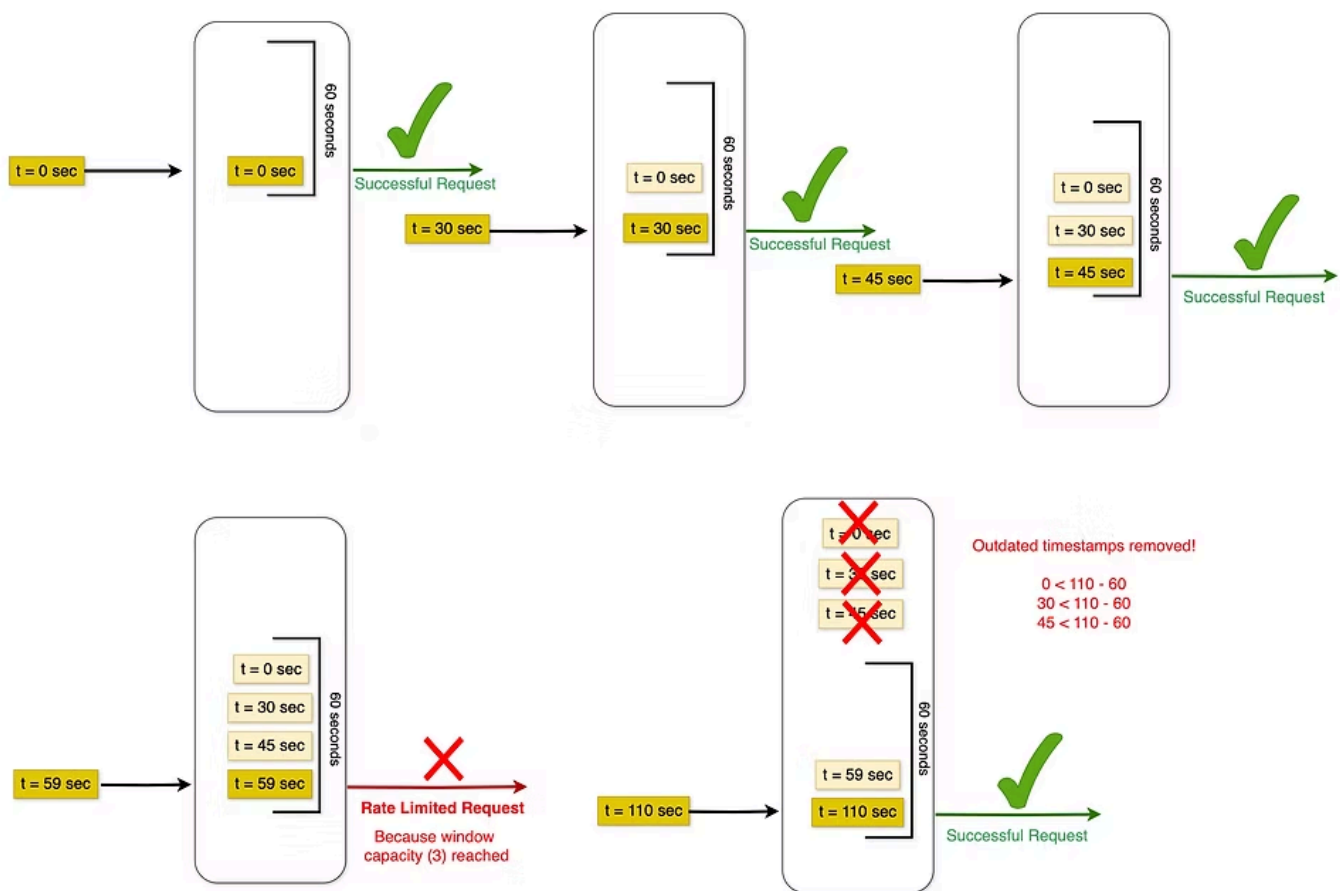
How It Works

Follow the diagram chronologically from $t = 0$ till $t = 110$ sec to understand how the algorithm works.

1. When a request comes in, first look back **60 seconds** using the sliding window.
2. Remove any requests outside this window.
3. Count the requests within the window and add **1** for the new request.
4. If within the limit, the request passes through.
5. If it exceeds the limit, the request is **rate-limited**.

Example

At $t = 59$ sec, the window's capacity exceeds **3**, so the **59 sec request is rate-limited**. However, its timestamp is still added to the queue because the system tracks all requests, whether accepted or rejected.



Note

The Sliding Window Log Algorithm is precise but uses more memory since even rejected requests' logs (timestamps) are stored.

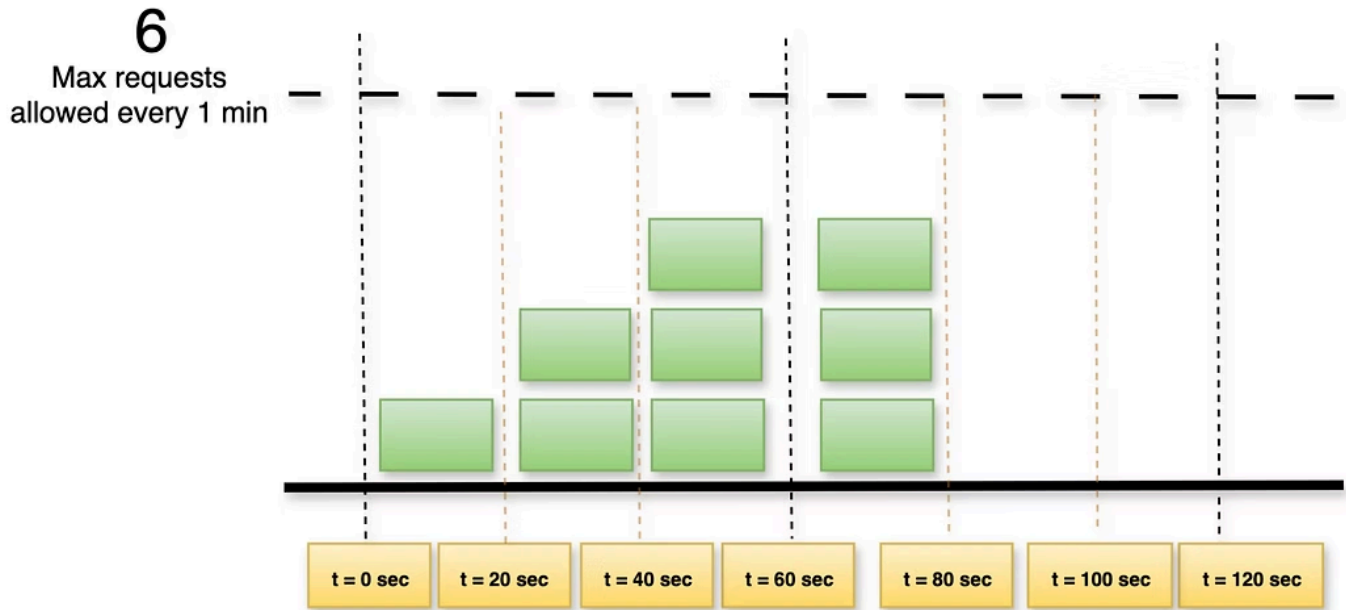
DEEP DIVE INSIGHTS :Sliding Window Counter Algorithm

The best way to understand this algorithm is by walking through examples.

Consider the following scenario:

- From **0 to 60 sec**, we get **6 requests**:
 - 1st request:** [0 sec, 20 sec]
 - 2 requests:** [20 sec, 40 sec]
 - 3 requests:** [40 sec, 60 sec]

- From **60 to 120 sec**, we get **3 requests**:
 - **All 3 requests**: [0 sec, 20 sec]
- **Max allowed requests** = 6 per minute.



Fixed Window Algorithm

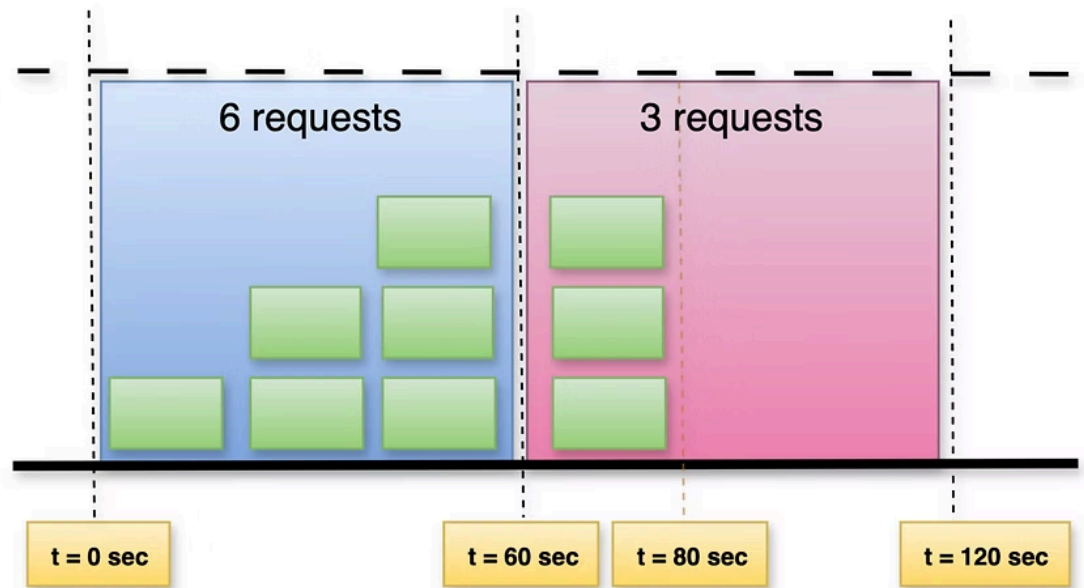
Let's see how the **Fixed Window Algorithm** works in this case:

- At **80 sec**, we have **3 requests in the second window**.
- Since **3 < 6** (allowed limit), no request will be rate-limited.

Issue:

- This is incorrect because **bursts of traffic at the edges** are not rate-limited.

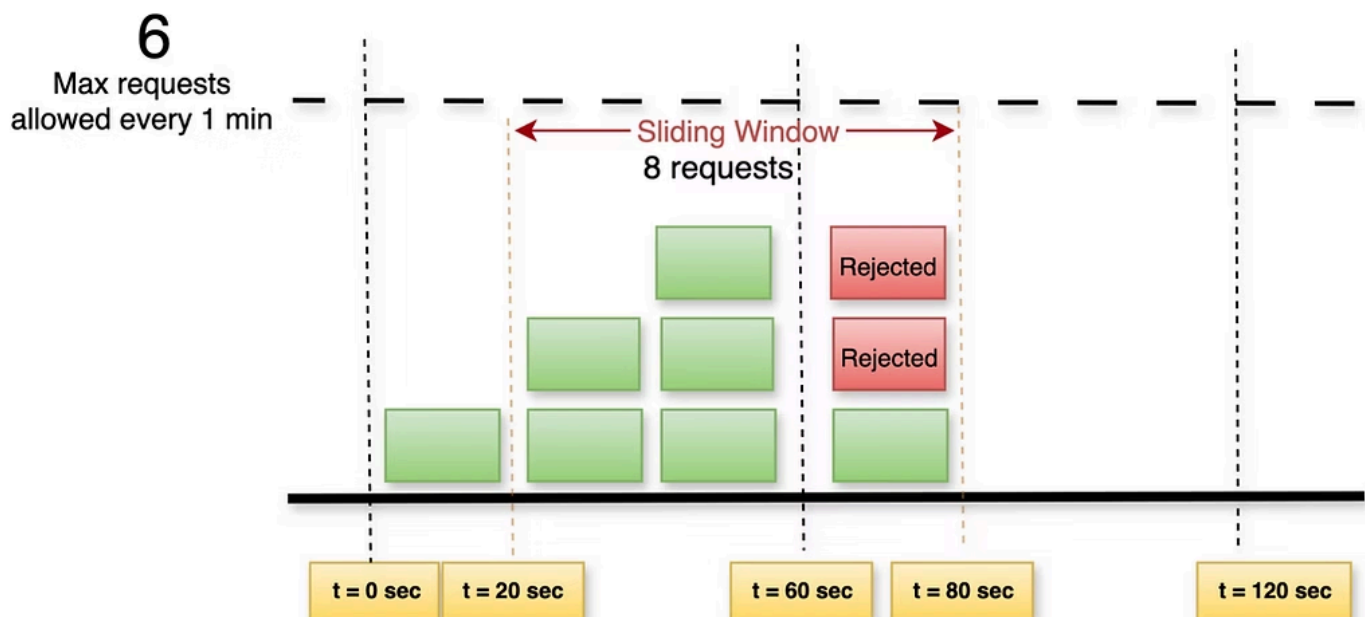
6
Max requests
allowed every 1 min



Sliding Window Logs Algorithm

Let's see how the **Sliding Window Logs Algorithm** works:

- At **80 sec**, we look back **60 sec** using a sliding window.
- There are **8 requests** in this window.
- The limit is **6**, so the **last 2 requests are rejected**.



Sliding Window Counter Algorithm

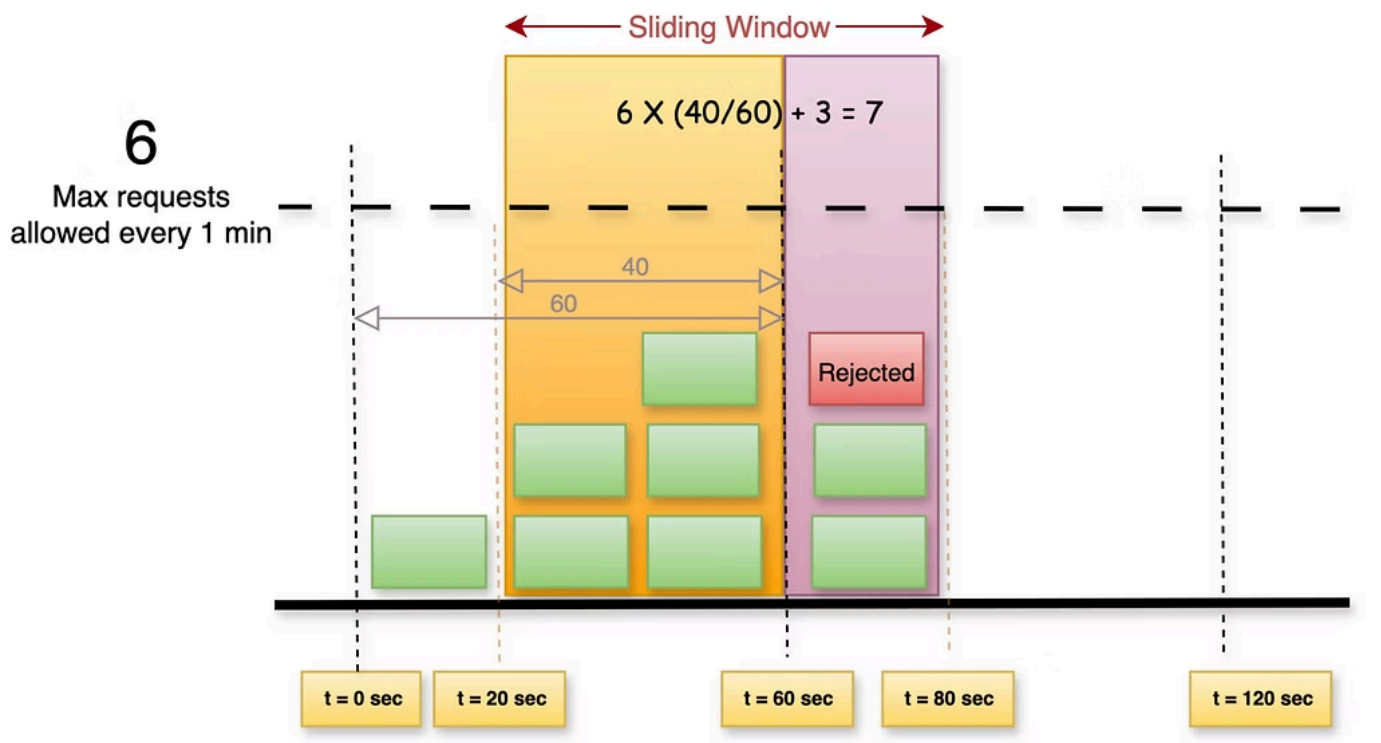
Let's see how the **Sliding Window Counter Algorithm** works:

- At **80 sec**, we look back **60 sec** using a sliding window.
- **Current window (second window): 3 requests**
- **Previous window (first window): 6 requests**
- The **sliding window overlaps** with the first window from **20 sec to 60 sec**.
- In this overlapped part, there are **5 requests**.

Approximation:

Instead of counting the exact requests in the overlapped part, we assume requests are **evenly distributed** in the first window.

- **Average requests in overlapped part =**
 $\text{Total requests in first window (6)} \times \text{Overlap percentage (40/60)} = 4 \text{ requests}$
- **Total requests in the sliding window =**
 $3 \text{ (current window)} + 4 \text{ (average from overlapped part)} = 7$
- **Limit exceeded ($7 > 6$), so the last request is rejected.**



Benefits

- Unlike **Sliding Window Log**, we don't need to store all request timestamps, making it **memory efficient**.
- Using **averages is faster** than counting all timestamps in the sliding window.
- Minor errors may occur, but it usually works well.

Fun Fact:

Cloudflare's experiments show that **99.997% of 400 million requests** are correctly processed, with only **0.003% errors**.

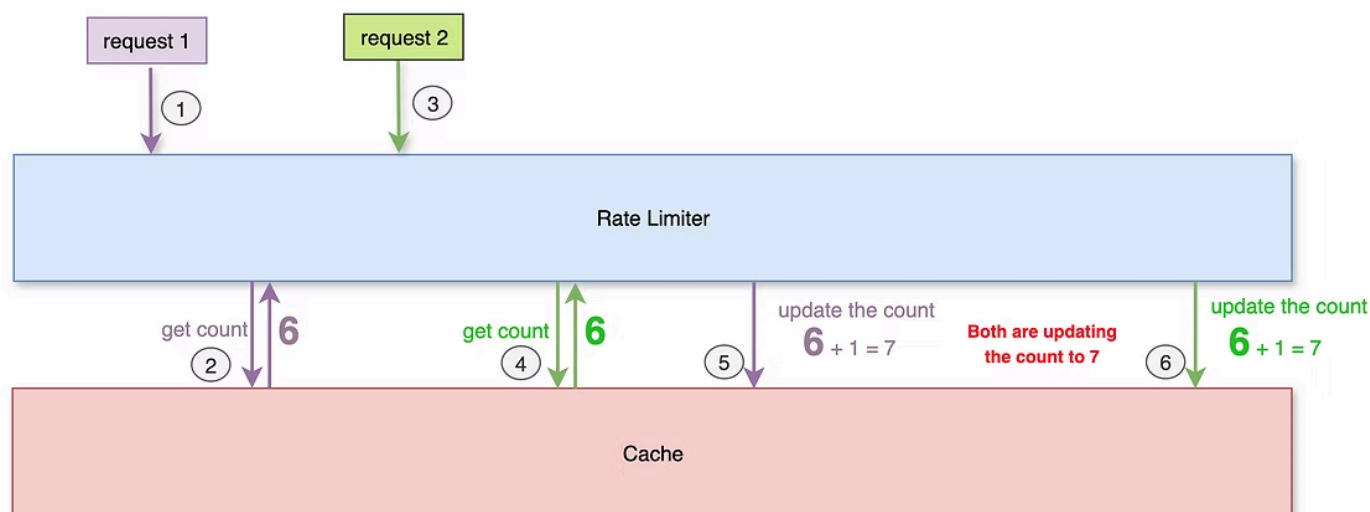
DEEP DIVE INSIGHTS :Handling Race condition

So far, we've seen rate-limiting as a single component. But in real life, we don't just have one rate-limiter; we have multiple instances. Plus, there are concurrent (same time) requests happening.

Scenario

Let's assume this scenario:

1. A limiter receives two requests from the same user at the same time.
2. The first request reads the counter value (**6**).
3. Then, it increments the value by one (**6 + 1**).



Now, imagine that in the middle of this process (between step 3 and step 4), another request comes in. The first request hasn't updated the counter yet, so this second request also reads the counter value as **6**.

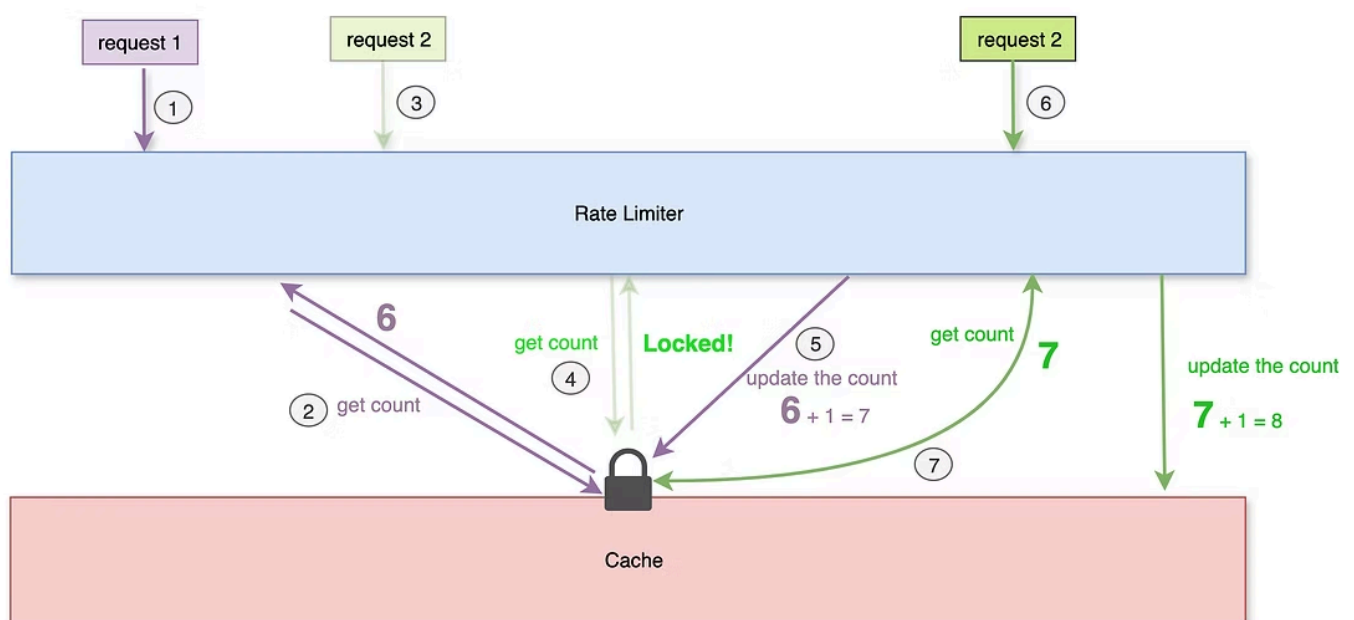
After both requests update, they each write back **7**, but the correct value should be **8**.

This issue is called a **race condition** because the two requests are racing to read and update the same value.

Solution: Using Locks

To solve this, we can use **locks**. A lock ensures that while one request is processing, other requests have to wait. Once the first request is done and releases the lock, the next request can proceed.

- The second request will now read the updated value (**7**) and then correctly update it to **8**.



Using locks can slow down the system a bit. There are other methods to handle this, like using **Lua scripts**, but we'll discuss those another time.